TERRAFORM COMPREHENSIVE NOTES

BY SANGYA SAGAREEKA

Table of Contents

- What is Infrastructure as Code?
- What is Terraform?
- Why do we use Terraform?
- What is HCL?
- Important Terminologies in Terraform
  - Providers
  - Resources
  - Modules
  - Outputs
  - State
  - Backend
- Components of HCL
  - Blocks
  - Arguments
  - Variables
  - Expressions and Interpolation
  - Blocks Within Blocks

    - Comments
- What is the State file in Terraform? What's the importance of it?
- What is Desired and Current State?
- Summary
- TASK: Install Terraform
- TASKS
    - TASK 1: Create a Terraform script with Blocks and Resources.
    - TASK 2: Create a Resource Block for an nginx docker image and docker container.
- Summary

What is Infrastructure as Code (IaC)?

- **Definition**: Infrastructure as Code (IaC) is the practice of managing and provisioning IT infrastructure using machine-readable definition files, rather than manual processes.
- **Purpose**: Automates the configuration, deployment, and management of infrastructure components—reducing human error and increasing efficiency.
- **Managed Resources**: Includes networks, virtual machines, load balancers, storage, and connectivity.
- **Languages Used**: Typically written in **YAML**, **JSON**, or domain-specific languages like **HCL** (used by Terraform).
- **Version Control**: IaC configurations are stored in source control systems (e.g., Git), enabling **change tracking**, **collaboration**, and **rollbacks**.
- **Modular Architecture**: Infrastructure can be broken into reusable and composable modules for greater flexibility and scalability.
- **Benefits**:
    - Consistency across environments
    - Faster, repeatable deployments
    - Improved collaboration between development and operations teams
    - Easier auditing and compliance

What is Terraform?

- **Terraform** is an open-source **Infrastructure as Code (IaC)** tool developed by **HashiCorp**.
- **Purpose**: Automates the **provisioning, configuration, and management** of infrastructure across multiple cloud providers and on-prem platforms.
- **Declarative Language**: Uses **HashiCorp Configuration Language (HCL)** or **JSON** to define the desired state of infrastructure resources.
- **Key Features**:
  - Platform-agnostic: Supports major cloud providers (e.g., AWS, Azure, GCP) and many other services.
  - Dependency management: Understands resource dependencies and provisions them in the correct order.
  - Modular design: Promotes reusable, composable configuration components.
  - Version control friendly: Configuration files can be stored and managed via Git or other VCS systems.
- **Benefits**:
  - **Repeatable deployments** with consistent environments
  - **Infrastructure versioning** and rollback capabilities
  - Improved **collaboration** and **auditability** through code reviews
  - Faster provisioning with minimal manual effort

Why Use Terraform?

- **Cloud Resource Provisioning**:
  Automates the provisioning and management of compute, storage, networking, and application services across cloud providers like **AWS**, **Azure**, **GCP**, and others.

- **Multi-Cloud & Hybrid Cloud Management**:
  Enables unified management of infrastructure across multiple cloud platforms, making it ideal for **multi-cloud** and **hybrid-cloud** environments.

- **Version Control & Collaboration**:
  Infrastructure configurations can be stored in systems like **Git**, allowing for:

  - Team collaboration
  - Change tracking
  - Rollbacks to previous infrastructure states

- **CI/CD & Automation**:
  Easily integrates with **CI/CD pipelines** to automatically apply infrastructure changes alongside application deployments.

- **Dependency Management**:
  Automatically manages **resource dependencies**, ensuring resources are created, updated, or destroyed in the correct order.

- **Strong Ecosystem & Community**:
  Backed by a large, active community and a wide range of **provider plugins**, enabling integration with hundreds of services and tools.

What is HCL?

- **HCL** stands for **HashiCorp Configuration Language**, developed by **HashiCorp**.
- It is the **declarative language** used by **Terraform** to define infrastructure as code.
- Designed to be **human-readable and intuitive**, making it easier for users to express infrastructure requirements.
- Provides a **structured syntax** to define:
  - Resources
  - Configuration parameters
  - Relationships and dependencies
- HCL files typically use the **`.tf` extension** and represent the **desired state** of infrastructure.
- Benefits:
  - Clear and concise syntax
  - Purpose-built for infrastructure provisioning
  - Supports variables, modules, and interpolation

Important Terminologies in Terraform

🔌 Providers

- A **provider** is a plugin that enables Terraform to interact with specific cloud platforms or services.
- It acts as the **bridge** between Terraform and your desired infrastructure (e.g., AWS, Azure, GCP).
- Each provider supports a unique set of resources and configuration options.

Example:

```
provider "aws" {
  region = "us-east-1"
}
```

🛠️ Resources

- A **resource** defines a specific piece of infrastructure that Terraform will **create, update, or delete**.
- Resources include **VMs**, **networks**, **databases**, **load balancers**, etc.
- Each resource has a configuration block that specifies its **type**, **name**, and **properties**.

Example:

```
resource "aws_instance" "my_instance" {
  count         = 5
  ami           = "<ami-id>"
  instance_type = "t2.micro"

  tags = {
    Name = "terra-instance-server"
  }
}
```

📦 Modules

- A **module** is a reusable, self-contained block of Terraform configuration that encapsulates multiple resources.
- Modules promote:
  - **Modularity** – break complex configs into logical units
  - **Reusability** – use the same module across environments or projects
  - **Maintainability** – simplify updates and reduce duplication
- Modules can be **local**, **shared internally**, or **pulled from public registries**.

Example:

```
module "dev-app" {
  source              = "./modules/my-app"
  my_environment      = "dev"
  ami                 = "<ami-id>"
  instance_type       = "t2.micro"
  instance_name       = "demo"
  bucket_name         = "demo-bucket-my-app"
  dynamo_table_name   = "demo-table-my-app"
}
```

📤 Outputs

- **Outputs** are used to extract and display values from your Terraform configuration.
- Commonly used to expose:
  - IP addresses
  - Resource IDs
  - URLs

- Useful for **integration with external systems**, **manual inspection**, or **chaining modules**.

Example:

```
output "my_ec2_ip" {
  value = aws_instance.my_instance[*].public_ip
}
```

📄 Terraform State

- **Terraform state** is a file that tracks the **actual state** of infrastructure managed by Terraform.
- It acts as a **source of truth** for resource configurations, attributes, dependencies, and relationships.
- Terraform uses this state to:
  - Determine what exists
  - Track changes
  - Plan updates
  - Safely create, update, or delete resources
- The state is typically stored in a local file named `terraform.tfstate`, but can be stored remotely (e.g., in **S3**, **Azure Blob**, **Terraform Cloud**) for **team collaboration** and **locking**.

Example Context

```
provider "aws" {
  region = "us-east-1"
}

resource "aws_s3_bucket" "my_bucket" {
  bucket = "cf-s3-..."
  acl    = "private"
}

resource "aws_s3_bucket_public_access_block"
"example" {
  bucket = aws_s3_bucket.my_bucket.id
}

resource "aws_s3_bucket_logging" "example" {
  bucket         = aws_s3_bucket.my_bucket.id
  target_bucket  = aws_s3_bucket.my_bucket.id
  target_prefix  = "logs/"
}

resource "aws_s3_bucket_notification" "example" {
  bucket = aws_s3_bucket.my_bucket.id

  lambda_function {
    lambda_function_arn =
aws_lambda_function.my_lambda_function.arn
    events              = ["s3:ObjectCreated:*"]
    filter_prefix       = "uploads/"
    filter_suffix       = ".jpg"
  }
}
```

- In this example, Terraform tracks the creation and configuration of the **S3 bucket**, **access blocks**, **logging**, and **notifications**.

- The **state file** maintains the mappings between the configuration and real-world AWS resources to ensure accurate provisioning and updates.

🗂️ Backend in Terraform

- A **backend** in Terraform defines **where and how** the **Terraform state file** is stored and managed.
- Backends enable:
  - **Remote state storage** (e.g., AWS S3, Azure Blob, Terraform Cloud)
  - **State locking** to prevent concurrent modifications
  - **Collaboration** across teams with shared access to the state
- By default, Terraform stores the state **locally** (`terraform.tfstate`), but using a **remote backend** is recommended for team environments.

Example: Remote Backend (Terraform Cloud)

```
terraform {
  backend "remote" {
    organization = "example_corp"

    workspaces {
      name = "my-app-prod"
    }
  }
}
```

✅ Common Remote Backends:

- **Terraform Cloud / Enterprise**
- **Amazon S3** (with optional DynamoDB for locking)
- **Azure Blob Storage**
- **Google Cloud Storage (GCS)**
- **Consul**

🧩 Components of HCL

📦 Blocks

- The core structural element of HCL.
- Enclosed in **curly braces `{}`**, blocks define the purpose and scope of a configuration.
- Common block types: `provider`, `resource`, `variable`, `output`, `module`.

Syntax:

```
<block_type> <resource_type> <resource_name> {
  arguments
}
```

Example:

```
resource "aws_instance" "example" {
  instance_type = "t2.micro"
  ami           = "ami-12345678"
  key_name      = "my-key-pair"
  subnet_id     = “subnet-12345678" }
```

⚙️ Arguments

- **Key-value pairs** used within blocks to define specific settings.
- Syntax: `key = value`
- They configure the behavior or properties of a resource, provider, or module.

Example:

```
instance_type = "t2.micro"
region        = "ap-south-1"
```

🔧 Variables

- Used to **parameterize configurations**, making them dynamic and reusable.
- Defined using the `variable` block.
- Variables can be:
  - Given a **default** value
  - Left as **required input**
  - Typed (e.g., `string`, `number`, `bool`, `list`, etc.)

Example:

```
variable "region" {
  type    = string
  default = "ap-south-1"
}
```

🧠 Expressions & Interpolation in HCL

🔄 Expressions

- HCL allows **expressions** to dynamically compute or manipulate values.
- Common use cases include:
  - **String concatenation**
  - **Arithmetic operations**
  - **Conditionals (`? :`)**
  - **Logical comparisons**

Example:

```
count = var.enable ? 1 : 0
name  = “server-${var.env}"
```

🧬 Interpolation

- **Interpolation** lets you reference variables, resource attributes, or functions using the syntax:
  `${ <expression> }`
- In Terraform 0.12 and later, interpolation is simplified and often does **not** require `${}` for simple references.

Modern Style Example:

```
resource "aws_instance" "example" {
  instance_type = var.instance_type
  ami           = data.aws_ami.ubuntu.id
}
```

Classic Style (still valid):

```
ami = "${data.aws_ami.ubuntu.id}"
```

✅ Use Cases

- Referencing variable values: `var.instance_type`
- Accessing data sources: `data.aws_ami.ubuntu.id`
- Concatenating values: `"app-${var.env}-web"`
- Conditional logic: `var.env == "prod" ? "t3.large" : "t3.micro"`

🧱 Blocks Within Blocks

- HCL supports **nested blocks**, allowing you to define **hierarchical** configurations.
- This is useful for expressing **complex resource relationships** (e.g., a VPC with subnets, a resource with inline settings).
- Child blocks inherit context from their parent and are specific to the enclosing block's type.

Example:

```
provider "aws" {
  region = "us-east-1"
}

resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"

  tags = {
    Name = "main-vpc"
  }

  subnet {
    cidr_block        = "10.0.1.0/24"
```

```
    availability_zone = "us-east-1a"
  }
}
```

- In this example:
  - The `subnet` block is **nested** inside the `aws_vpc` resource block.
  - It represents configuration tightly coupled with the VPC.

💬 Comments in HCL

- Comments are used to **document**, **explain**, or **annotate** code within Terraform configuration files.
- HCL supports two types of comments:

✅ Single-line Comment

```
# This is a single-line comment
```

✅ Multi-line Comment

```
/*
This is a multi-line comment.
It can span multiple lines.
*/
```

📝 **Tip**: Use comments to improve maintainability, especially in shared or team-managed codebases.

📄 What is the Terraform State File?

- The **Terraform state file** (`terraform.tfstate`) is a **JSON file** that records the **actual state** of infrastructure managed by Terraform.
- It includes metadata such as:
  - Resource attributes (e.g., instance ID, IP address)
  - Dependencies and relationships
  - Module paths and resource hierarchy

🧠 Importance of the State File

- Serves as the **single source of truth** for Terraform to:
  - Understand what has been provisioned
  - Track infrastructure changes over time
  - Generate accurate **execution plans** (`terraform plan`)
- Ensures **idempotency**: repeated runs yield consistent results.
- Enables:
  - **Accurate updates** (e.g., change only what's necessary)
  - **Collaboration** when using **remote state backends** with locking
  - **Resource dependency management**
  - **Rollbacks** and auditing (with state versioning)

🔐 **Best Practice**: Store the state file in a **remote backend** (e.g., AWS S3 with DynamoDB locking) for team collaboration and disaster recovery.

🔄 Desired State vs. Current State

✅ Desired State

- Defined in your **Terraform configuration files** (`.tf`).
- Represents the **infrastructure you want** to provision or maintain.
- Specifies resource types, properties, count, relationships, etc.

📌 Current State

- Captured in the **state file**.
- Reflects the **actual, real-world infrastructure** as known to Terraform.
- Tracks what has been created, modified, or destroyed.

During a `terraform plan`, Terraform **compares the Desired State with the Current State** to determine what actions to perform (create, update, delete) to reach the desired end state.

🚀 Task: Install Terraform on AWS EC2 (Ubuntu/Debian-based)

✅ Step 1: Launch EC2 and SSH into it

1. Log in to your **AWS Console**.
2. Launch a **t2.micro** EC2 instance with a **Ubuntu AMI** (e.g., Ubuntu 22.04 LTS).
3. Allow **SSH (port 22)** in the security group.
4. Connect to the instance using your terminal:

```
ssh -i <your-key.pem> ubuntu@<your-ec2-public-ip>
```

✅ Step 2: Add the HashiCorp GPG Key and Repository

```
sudo apt-get update

# Add HashiCorp GPG key
wget -O- https://apt.releases.hashicorp.com/gpg \
| sudo gpg --dearmor -o /usr/share/keyrings/
hashicorp-archive-keyring.gpg

# Add the official Terraform repo
echo "deb [signed-by=/usr/share/keyrings/hashicorp-
archive-keyring.gpg] \
https://apt.releases.hashicorp.com $(lsb_release -cs)
main" \
| sudo tee /etc/apt/sources.list.d/hashicorp.list
```

✅ Step 3: Install Terraform

```
sudo apt update && sudo apt install terraform -y
```

✅ Step 4: Verify Installation

```
terraform --version
```

You should see the installed Terraform version, confirming a successful setup.

✅ Summary

Terraform, developed by **HashiCorp**, stands as a powerful and widely adopted **Infrastructure as Code (IaC)** tool. It allows teams to **define**, **provision**, and **manage** infrastructure using a **declarative configuration language**, supporting multiple cloud platforms and on-premises environments.

At the heart of Terraform lies **HCL (HashiCorp Configuration Language)**—a human-readable, declarative language purpose-built for infrastructure configuration. Its simplicity and clarity enable both new and experienced users to describe resources, relationships, and logic effectively.

In this guide, we explored:

- The **core concepts of IaC**
- Terraform's key components: **Provider**, **Resource**, **Module**, **Output**, **State**, and **Backend**
- The structure and elements of HCL: **Blocks**, **Arguments**, **Variables**, **Expressions**, **Interpolation**, and **Nested Blocks**
- Step-by-step instructions to **install Terraform** on an AWS EC2 instance

By embracing Terraform and HCL, teams can achieve **automation**, **consistency**, and **scalability**—empowering modern DevOps practices and revolutionizing how infrastructure is managed.

🚀 Terraform Task: Docker Provider with Nginx Container

This guide walks you through creating a Terraform script using **Blocks** and **Resources** to deploy an **Nginx Docker container** on an EC2 instance.

✅ TASK 1: Set Up Terraform with Docker Provider

🔷 Step 1: Launch EC2 & SSH into Instance

- Login to **AWS Console**.
- Launch an **EC2 instance** (Ubuntu preferred).
- Enable **SSH access** via Security Group.
- Connect using:

```
ssh -i <your-key.pem> ubuntu@<your-ec2-public-ip>
```

🔷 Step 2: Create the Terraform Configuration File

Create `terraform.tf` with the required provider block:

```
vim terraform.tf
```

Paste the following:

```
terraform {
  required_providers {
    docker = {
      source  = "kreuzwerker/docker"
      version = "~> 2.21.0"
    }
  }
}
```

ℹ️ `kreuzwerker/docker` is the Terraform provider for Docker.

🔷 Step 3: Add Docker Provider Block

```
provider "docker" {}
```

✅ TASK 2: Create Docker Resources (Image + Container)

🔷 Step 1: Define Docker Image Resource in `main.tf`

```
resource "docker_image" "my_nginx" {
  name         = "nginx:latest"
  keep_locally = false
}
```

🔷 Step 2: Define Docker Container Resource in `main.tf`

```
resource "docker_container" "my_nginx_container" {
  image = docker_image.my_nginx.name
  name  = "nginx-container"

  ports {
    internal = 80
    external = 80
  }
}
```

🔷 Step 3: Install Docker (if not already installed)

```
sudo apt-get update
sudo apt-get install docker.io -y
sudo chown $USER /var/run/docker.sock
docker --version
```

🔷 Step 4: Install Terraform (if not already installed)

Refer to the previous task.

🔷 Step 5: Initialize Terraform Project

```
terraform init
```

🔷 Step 6: Preview Execution Plan

```
terraform plan
```

🔷 Step 7: Apply Terraform Configuration

```
terraform apply
```

- Confirm with `yes` when prompted.

🔷 Step 8: Verify Docker Container is Running

```
docker ps
```

- You should see `nginx-container` up and running.

🔷 Step 9: Open Nginx in Browser

- Visit your EC2 public IP in the browser (port **80**) to see the **Nginx default page**.

🔷 Step 10: Destroy Terraform Resources

```
terraform destroy
```

- Confirm with `yes` to delete resources.

This will remove the Docker container and image created by Terraform.

✅ Summary

In this guide, we explored key building blocks of Terraform—**Blocks**, **Resources**, and **Providers**—which form the foundation of writing and managing infrastructure as code.

- 🔷 **Blocks**: The primary structural units in HCL used to define configuration settings. Common blocks include `resource`, `provider`, `variable`, `output`, and `module`.
- 🔷 **Resources**: Represent the actual infrastructure components (like VMs, databases, containers). Declared within `resource` blocks, they define what Terraform should create, update, or destroy.
- 🔷 **Providers**: Act as a bridge between Terraform and the target infrastructure (e.g., AWS, Azure, Docker). They are declared using `provider` blocks and are responsible for managing the lifecycle of resources.

Together, these elements allow you to describe and provision infrastructure declaratively and reproducibly.

In our hands-on example, we applied these concepts by:

- Writing a **Terraform Docker script**
- Using **resource blocks** to pull an Nginx Docker image and run it inside a container
- Managing all configurations through structured blocks

By leveraging Terraform's modular, declarative approach, we gain **consistency**, **scalability**, and **automation** in infrastructure management.